

NOTES ON IMMERSED BOUNDARY METHOD

Junior Year

THOMAS LI

Undergraduate Student at New York University Shanghai

jl15535@nyu.edu

Contents

1	Introduction	1
1.1	Mathematical Formulation	1
1.2	Numerical Methods	1
1.3	The Interpolation Function	2
1.4	Discrete Delta Function	2
2	Penalty Immersed Boundary Method (PIB)	4
2.1	Rigid Body pIB	4
2.1.1	Implementation : Circle in Flow	4

SECTION 1

Introduction

The Immersed Boundary (IB) method is a general numerical framework for studying fluid-structure interaction. It was originally developed by Charles Peskin to simulate blood flow in the heart, but has since been applied to a wide variety of problems in biofluid dynamics and beyond.

The fundamental approach of the IB method is to approximate the fluid equations on a fixed Eulerian grid (usually Cartesian), while tracking the flexible immersed boundary using a set of Lagrangian points that move with the fluid. The interaction between the fluid and the structure is mediated by the Dirac delta function, which allows forces to be spread from the Lagrangian boundary to the Eulerian fluid grid, and velocities to be interpolated from the fluid grid to the boundary points. This avoids the need for complex body-fitted mesh generation, making the method particularly suitable for problems with large structural deformations or complex geometries.

SUBSECTION 1.1

Mathematical Formulation

The equations of motion for the fluid are the incompressible Navier-Stokes equations:

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla p + \mu \nabla^2 \mathbf{u} + \mathbf{f} \quad (1.1)$$

$$\nabla \cdot \mathbf{u} = 0 \quad (1.2)$$

where $\mathbf{u}(\mathbf{x}, t)$ is the fluid velocity, $p(\mathbf{x}, t)$ is the pressure, ρ is the fluid density, and μ is the dynamic viscosity. The term $\mathbf{f}(\mathbf{x}, t)$ represents the force density applied by the immersed boundary on the fluid.

The interaction between the fluid and the immersed boundary is described by the following integral equations:

$$\mathbf{f}(\mathbf{x}, t) = \int_{\Gamma} \mathbf{F}(s, t) \delta(\mathbf{x} - \mathbf{X}(s, t)) ds \quad (1.3)$$

$$\frac{\partial \mathbf{X}}{\partial t}(s, t) = \mathbf{u}(\mathbf{X}(s, t), t) = \int_{\Omega} \mathbf{u}(\mathbf{x}, t) \delta(\mathbf{x} - \mathbf{X}(s, t)) d\mathbf{x} \quad (1.4)$$

Here, $\mathbf{X}(s, t)$ denotes the position of the immersed boundary parameterized by the curvilinear coordinate s , and $\mathbf{F}(s, t)$ is the force density on the boundary. The Dirac delta function $\delta(\mathbf{x})$ mediates the interaction between the Eulerian fluid variables and the Lagrangian boundary variables.

SUBSECTION 1.2

Numerical Methods

The numerical implementation typically involves answering the following on each time step:

1. **Force computation:** Calculate the boundary force $\mathbf{F}(s, t)$ from the boundary configuration $\mathbf{X}(s, t)$ using the constitutive laws (e.g., Hooke's law).
2. **Force spreading:** Spread the boundary force to the fluid grid using the discretized delta function to obtain $\mathbf{f}(\mathbf{x}, t)$.

3. **Fluid solver:** Solve the Navier-Stokes equations on the Eulerian grid to update the velocity field $\mathbf{u}$ and pressure p . This is often done using a projection method or a fractional step method.
4. **Velocity interpolation:** Interpolate the new fluid velocity to the boundary points applied to update their positions $\mathbf{X}(s, t)$.

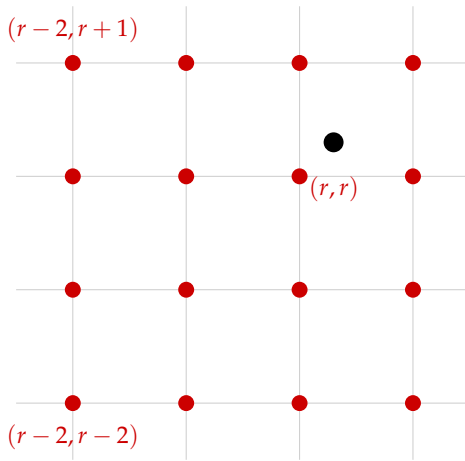
SUBSECTION 1.3

The Interpolation Function

Given the material position X we want to obtain the velocity of fluid at that point.¹ This process is directly interpreted by

¹Note that X usually doesn't lies on the grid point.

$$\mathbf{u}(\mathbf{X}(r, s, t), t) = \int \mathbf{u}(\mathbf{x}, t) \delta(\mathbf{x} - \mathbf{X}(r, s, t)) d\mathbf{x} \quad (1.5)$$



However in the discrete case we would need the discrete Delta Function.

In the figure on the left, black point is the material point and red points are the grid points. The δ function average all the velocity at the red points.

Something to pay attention is that in MATLAB, the convention for a matrix to represent a grid of point is to have lay points with same x coordinat in the **same row**. This may be different from our intuition. However, This is reasonable since we normally define the matrix for storing points to have size

$$N_x \times N_y$$

So it has N_x rows.

SUBSECTION 1.4

Discrete Delta Function

A commonly used discretized delta function uses a 4×4 grid. Thus the 1D delta function $\phi(x)$ should support on four points. We have the following postulates:

1. $\phi(r)$ is continuous for all real r
2. $\phi(r) = 0$ for $|r| \geq 2$
3. The following gurantee there is no difference between odd index grid point and even index grid point. Since they never exchange information between each others

$$\sum_{j \text{ even}} \phi(r - j) = \sum_{j \text{ odd}} \phi(r - j) = \frac{1}{2} \text{ for all real } r$$

4. The delta function should be centered at the origin

$$\sum_j (r - j)\phi(r - j) = 0 \text{ for all real } r$$

5. The L_2 norm should be given.

$$\sum_j (\phi(r - j))^2 = C \text{ for all real } r$$

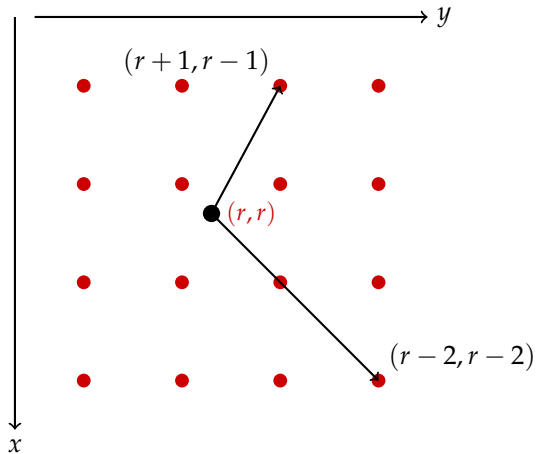
Here $r \in [0, 1)$. This is a standard **4 point delta function**. We can also have 6 points delta function for higher accuracy. C can be determined by setting $r = 0$. We get

$$\phi(r) = \frac{3 - 2r + \sqrt{\Delta}}{8} \quad \phi(r + 1) = \frac{3 - 2r - \sqrt{\Delta}}{8}$$

and

$$\phi(r - 1) = \frac{1 + 2r + \sqrt{\Delta}}{8} \quad \phi(r - 2) = \frac{1 + 2r - \sqrt{\Delta}}{8}$$

Here $\Delta = 1 + 4r - 4r^2$.



However, in MATLAB, grid points are stored in a particular way. Entries with same x coordinate is in same row and $(0,0)$ is at the upper-right corner.

Since the argument for ϕ is the position vector of grid point. The value for point $(r - 2, r - 2)$ is actually at the lower right as shown in the picture on the left. ²

²I made a mistake here when writing the function `twoD_delta_function`. The order is incorrect. Time: 251212

SECTION 2

Penalty Immersed Boundary Method (PIB)

The Penalty Immersed Boundary Methods is a simple approach try to solve fluid-structure interaction with material bounadary that has mass.

SUBSECTION 2.1

Rigid Body pIB

One kind of problem pIB can simulates is a rigid body in flow. The key of pIB is to consider two sets of Material Points, denoted as $X(q, s, t)$ and $Y(q, s, t)$. Here q, r are Lagrangian Coordinate and t is time. X is the true boundary points that interact masslessly with the flow, and def is called the **Ghost Mass**. It doesn't interact with the flow but only with the Material points X through an imaginary spring.

Explanation 1

1. At the start, the Material points' position is updated using fluid velocity interpolated at X .

```
1 X_half_top = X_tube_top + dt / 2 *
   interpolation(u, X_tube_top, Num_b, Nx, Ny,
               dx, dy);
```

2. The force acting to the fluid by the Material is modeled by a spring connecting the Ghost point and material point.

$$\mathbf{F}(q, r, t) = K(\mathbf{Y}(q, r, t) - \mathbf{X}(q, r, t)) \quad (2.1)$$

3. Then spread the force F to Eucledian space.
4. Numerically solve the time evolution of fluid using Navier Stokes solver.
5. Update the Material point position X .

2.1.1 Implementation : Circle in Flow

I will explain the principles for choosing the timestep dt without entering task planning mode as it's a conceptual question.

When choosing the timestep dt for an Immersed Boundary Method (IBM) simulation (especially one solving the Navier-Stokes equations with explicit or semi-implicit time integration), you generally need to satisfy several stability constraints. The chosen dt must be the minimum of the time steps required by each of these independent conditions.

Here are the primary principles and criteria you should refer to:

1. The Courant-Friedrichs-Lewy (CFL) Condition (Advective Stability) This is the most fundamental restriction for any explicit fluid solver. It ensures that fluid information doesn't move more than one grid cell per timestep. For a 2D simulation, the CFL condition is:

$$\Delta t_{\text{CFL}} \leq C_{\text{max}} \left(\frac{\Delta x}{|u|_{\text{max}}} + \frac{\Delta y}{|v|_{\text{max}}} \right)^{-1} \quad \text{or simply} \quad \Delta t_{\text{CFL}} \leq C_{\text{max}} \frac{\min(\Delta x, \Delta y)}{V_{\text{max}}}$$

Where:

V_{max} is the maximum expected velocity magnitude in the domain. C_{max} is the Courant number, which typically must be ≤ 1.0 for stability, but often a safer value like $0.2 \sim 0.5$

is chosen in IBM to account for explicit body forces. 2. Viscous Diffusion Stability Constraint If your Viscous term (the diffusion term $\mu \nabla^2 \mathbf{u}$) is solved explicitly, you have a strict parabolic stability limit:

$$\Delta t_{\text{visc}} \leq \frac{1}{2\nu} \left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2} \right)^{-1}$$

Where $\nu = \mu/\rho$ is the kinematic viscosity. (Note: If your Navier-Stokes solver uses an implicit method like Crank-Nicolson for the viscous terms, this specific restriction is bypassed, and only the CFL condition applies for the fluid).

3. Immersed Boundary Elastic Stability (The biggest restrictor) In the Immersed Boundary Method, the fluid and the boundary are coupled explicitly. The boundary exerts elastic restoring forces (spring forces K_{mem} and tether/penalty forces K_{tar}) on the fluid.

This explicit coupling introduces severe numerical stiffness. If Δt is too large, an elastic restoring force will over-correct the position of a boundary point, causing it to overshoot its equilibrium position in the next step, leading to explosive oscillations (unstable blow-up).

The stability limits for these explicit Hookean springs roughly scale as:

$$\Delta t_{\text{spring}} \sim \mathcal{O} \left(\frac{\Delta x}{\sqrt{K_{\text{mem}}/\rho}} \right)$$

and for the penalty force:

$$\Delta t_{\text{penalty}} \sim \mathcal{O} \left(\sqrt{\frac{m}{K_{\text{tar}}}} \right) \quad \text{or numerically} \quad \Delta t_{\text{penalty}} \propto \frac{\Delta x^2}{K_{\text{tar}}}$$

Practical Takeaway for IBM: Because of the explicit coupling of the stiff membrane forces, the timestep is almost always heavily dominated by your boundary stiffness. When you increased K_{tar} to 1000 and K_{mem} to 10 in your latest changes, you successfully compensated by drastically dropping your dt from 0.005 to 0.0001! This is exactly the right intuition.

Recommended Workflow for tuning dt: Start Conservative: Find a dt that works without blowing up (like your 0.0001). Monitor maximum velocity: Keep track of your maximum $\mathbf{u}$ in the domain to ensure your $\text{CFL} \approx \frac{V_{\text{max}} \Delta t}{\Delta x}$ stays well below 0.5. If the simulation blows up immediately (often outputting NaN everywhere): Your K_{tar} or K_{mem} is too stiff for your dt. You must either lower the stiffness or decrease dt. If the simulation blows up later in the run: This is usually a CFL violation where the fluid velocity locally accelerated too much, pushing the Courant number above 1.0. Decrease dt.